

Prompt Injection as the New SQL Injection: The Emergence of Attacks on AI Reasoning

Shuktaev Abzal
Dept. of Cybersecurity
Astana IT University
Astana, Kazakhstan
240293@astanait.edu.kz

Karabalin Bakhtiyar
Dept. of Cybersecurity
Astana IT University
Astana, Kazakhstan
240584@astanait.edu.kz

Orynbasar Sultan
Dept. of Cybersecurity
Astana IT University
Astana, Kazakhstan
242031@astanait.edu.kz

Olga Orlova
Department of Banking
Saint Petersburg State University of
Economics
St Petersburg, Russian Federation
Orlova.o@unecon.ru

Elena Aleksandrovna Korochkina
Head of the Department of Genetic and
Reproductive Biotechnology
St. Petersburg State University of
Veterinary Medicine
St. Petersburg, Russian Federation
e.kora@mail.ru

Sholpan Borashova
Senior Lecturer, IT Management and IT
Entrepreneurship Educational Programs
Astana IT University
Astana, Kazakhstan
Sholpan.Borashova@astanait.edu.kz

Abstract—The introduction of Large Language Models (LLMs) into enterprise application pipelines represents both a blessing and a curse — the same versatility in following instructions that enables conversational assistants, code copilots, and autonomous agents also erodes the distinction between trusted instruction and untrusted data. In this paper, we argue that Prompt Injection (PI) follows SQL Injection (SQLi) as a fundamental class of attack on machine learning systems. While SQLi leveraged the absence of distinction between code and data in relation queries, PI leverages the absence of separation between system instruction and contextual data in natural language reasoning. The target domain has shifted from syntactic parsing to semantic interpretation, and classic defenses — escaping, parameterization, and whitelisting — are insufficient because natural language lacks a formal grammar to sanitize. We perform a narrative analysis of relevant peer-reviewed and preprint publications within the period 2023–2026, structured around three major dimensions: (i) the taxonomy of direct and indirect prompt injection attacks, (ii) the tactics that PI employs to overcome defenses provided by Reinforcement Learning from Human Feedback (RLHF) and guardrails, and (iii) the elevated threat landscape of agentic LLM applications with tool capabilities. We formalize the attacker's strategic edge through a success rate model and offer comparative studies with SQLi, a severity scale for different modes of operation, and guidance for Chief Information Security Officers (CISOs). Our core thesis is that PI represents a paradigm shift, necessitating defense strategies beyond input sanitization toward workflow isolation and defensive LLMs.

Keywords—prompt injection; large language models; AI security; indirect prompt injection; agentic AI; adversarial machine learning; SQL injection analogy; defensive LLMs.

I. INTRODUCTION

A unique defect class was revealed for web applications during the first years of the new millennium—the inability of the data-driven engine to differentiate between userprovided values and executable instructions. SQL injection (SQLi) became a representative of that era since the flaw was independent of individual applications but manifested itself through an architectural feature of relational languages which concatenated user inputs into parsed statements [1], [2]. In the second decade, another collapse in the architecture of systems utilizing large language models (LLMs) takes place. Natural-language prompts combine in one token stream three types of texts—system instructions by developers, user inputs, and externally obtained contents—and do not provide a built-in

way to differentiate among them. As a result, the phenomenon known as prompt injection (PI) appeared in every family of existing models [3], [4], [5].

An analogy to SQL injection may be extended beyond rhetoric. Both kinds of flaws exploit trusted interpreters at the interface between the data and instructions, while the latter had become common and widespread before being discovered. Still, there is a critical difference between the two. SQLi acts at the syntactic level where an inserted quote or UNION SELECT exploits a parser with a formal and finite set of grammatical rules. PI is a semantic attack; a sentence inserted into a prompt manipulates a reasoning process that employs complex and high-dimensional statistical boundaries with an explicit tolerance for paraphrasing [6], [7].

Two additional observations on economics justify the analogy. First, the marginal cost of offense has fallen: writing a working prompt-injection payload does not require either reverse engineering or crafting exploits, only English, and payloads for a wide array of models are published publicly [8]. Second, LLM-integrated workflows improve attackers' operational capacities: an adversary used to relying on custom-built tooling to gain access to an email account, run a database query, or file a helpdesk request is now able to have an LLM agent do all of these actions for them using their tool access capabilities [9], [10]. As a result, the attacker is limited less by their technical capabilities and more by the workflow designer's.

In this paper we consolidate existing work on prompt injection (PI) from 2023 to early 2026. The use of a narrative-review methodology is justified by the novelty of the field, the terminological confusion it faces, and the lack of a single venue where the literature is collected. The contributions are (i) a structural comparison of SQL injection (SQLi) and PI that specifies what the analogy conserves and what changes; (ii) thematic analysis that considers direct and indirect PI attacks, as well as techniques that circumvent reasoning-layer defense and those that amplify attacker agency; (iii) a definition of attack success and attack risk for threat modeling purposes; and (iv) CISO recommendations.

The rest of the paper is structured as follows. Section II details the review methodology. Sections III through V contain the thematic analysis. Section VI explains why classical sanitization is inadequate and what has succeeded

it. Section VII presents a formal model. Section VIII examines enterprise preparedness.

II. REVIEW METHOD

Our analysis took a narrative as opposed to a systematic approach as the field of prompt-injection (PI) does not meet yet the pre-requisites of systematic/meta-analysis, being characterized by a lack of consistency in both evaluation protocols and metrics to measure success; furthermore, a non-negligible number of results in the literature are still only available as preprints or red-team reports posted outside academic platforms [11]. Hence, a narrative review remains the best tool to use whenever a coherent story is to be built on an emerging issue, as opposed to effect size estimation.

A. Sources and Search Strategy

Primary sources were identified using Google Scholar, arXiv cs.CR and cs.CL, ACM Digital Library, and IEEE Xplore. Searches included keywords “prompt injection”, “jailbreak”, “indirect prompt injection”, “LLM agent security”, “tool-using LLM”, and “adversarial prompts” with time restrictions set between January 2023 and March 2026. In addition to academic literature, technical documents from OWASP (particularly the LLM Top-10 series) [12] and security bulletins from vendors were consulted along with two industry red-team datasets, the release notes of which are considered gray literature.

B. Inclusion and Exclusion Criteria

The inclusion criteria consisted of studies that (i) introduced a specific attack, defense, or evaluation of the injection-class attacks in transformer-based language models; (ii) offered reproducible results or open-sourced artifacts; and (iii) were written in English. The exclusion criteria consisted of studies addressing only classical cases of adversarial examples targeting image or tabular models, papers discussing only the pre-training data-poisoning threat as part of the PI threat landscape, and opinion papers unsupported by empirical evidence. Overall, 84 primary sources were identified, out of which 28 are quoted in this paper.

C. Synthesis Approach

Every article in the reviewed body of work was analyzed on four axes: mode of the attack (direct, indirect, multistep), target attribute (content policies, reasoning integrity, invocation of the tool, extraction of data), evaluation context (monomodel, agentive, RAG-based), and, finally, potential defenses, when available. This analysis helped reveal a structure based on common codes in Sections III–V. The timeline of the reviewed articles is provided in Figure 1 below.

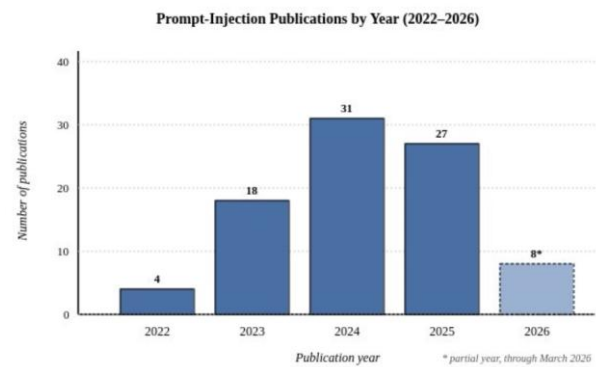


Fig. 1. Distribution of prompt-injection publications by year, 2022–2026. The corpus grows from 4 works in 2022 to 31 in 2024, with a plateau around 27 in 2025 reflecting consolidation rather than decline. Partial 2026 data through March are extrapolated.

III. THE LANDSCAPE OF PROMPT INJECTION

The most practical way of categorizing prompt injection exploits is based on the method by which the adversarial input is delivered to the model. This differentiation carries practical importance because it changes both the identity of the attacker and the necessary adjustment of trust assumptions, as well as the types of defenses that can apply.

A. Direct Prompt Injection

Direct prompt injection delivers the adversarial message via the interaction between the user and the model. Examples of this include the “ignore previous instructions” set of attacks that create content contrary to the policy of the system prompt [3]. Direct prompt injection has the most similarities with SQL injections because the attacker and the application user are one and the same person, and the attack is performed against the policy encoded into the system prompt. Modern attacks use role play templates, hypothetical situations, language diversion techniques, and encryption to overcome refined rejection mechanisms [14], [15].

While direct prompt injection attracts significant public attention because of the creation of images that could easily be shown on social media, its enterprise-level threat is somewhat mitigated by the fact that users could simply achieve the same effect without such an exploit: if a user asks for a recipe for a toxic material from the chatbot, he will find all that he needs from chemistry textbooks and web searches anyway.

B. Indirect Prompt Injection

Indirect Prompt Injection (PI) is the type of attack where the adversarial code is injected into some third-party data that needs to be processed by the large language model (LLM) — for example, an email, webpage, PDF, calendar event, or collaboration document [4], [16]. Usually, the user who poses the query does not know about the embedded command; the adversary and the LLM communicate with each other only through an intermediary. The inversion of the trust model becomes one of the critical characteristics of the modern attack surface. Unlike in the case of SQL injection (SQLi), in which there is a hostile actor using a legitimate database, in indirect PI, there is a good-faith individual whose assistant, driven by some document created earlier by someone else, acts against him.

Some empirical cases of the impact of indirect PI are: conversation data leakage through side channels based on image loading; altering summary generation to hide a fraudulent invoice entry; tricking a coding assistant to install a backdoored dependency in code review; and navigation hijacking of a web browser agent [5], [9], [17]. The example in Fig. 2 shows the steps of an attack sequence launched via an email-reading agent that resulted in data leakage from a single ingestion point.

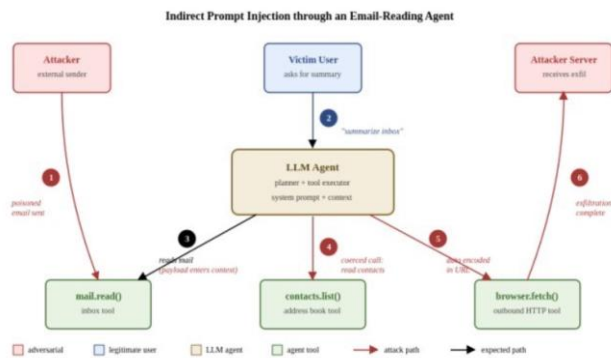


Fig. 2. Indirect prompt-injection chain through an email-reading agent. (1) The attacker sends an email containing an instruction disguised as signature text. (2) The victim asks the assistant to “summarize unread mail.” (3) The agent invokes the mail-read tool and ingests the attacker’s message as context. (4) The embedded instruction coerces the agent to query the contacts tool and (5) encode the result into a URL passed to the browsing tool, exfiltrating data to the attacker’s server. No alert is raised because every tool call is individually authorized.

C. Multi-turn and Stateful Attacks

There is a third type of attack that lies in between direct and indirect attacks. It is possible for an attacker to use the model’s memory within a single interaction or multiple interactions by feeding the model innocent-looking data that will turn into an attack only when a trigger is provided [18]. If the memory is maintained for a long period of time within the model, then this attack becomes persistent like cross-site scripting attacks.

IV. ATTACKING AI REASONING

Among all the defense-in-depth mechanisms, the one that most frequently cited as an example for modern LLMs is RLHF. In addition, there is an independent safety classifier and the specification of the prompt that states prohibited actions [19]. It was empirically shown in the study of PI attacks that all three defense layers exhibit graceful degradation behavior instead of failing closed; thus, under adversarial conditions, they respond to requests with non-zero but very low probability.

A. Why RLHF is Not a Security Boundary

In RLHF, the desired response output distribution is steered, but no secure kernel is created. The model doesn’t have any privileged area of weights that is absolutely out of bounds for the inputs; every prompt token is used in attention calculations in each layer. Thus, the sufficiently convincing context that reframes the prohibited activity in terms of translation, a question, or citing previous content can move the output distribution past the refusal barrier [15], [20]. Security properties based on probability, not impossibility, do not exist under adversarial conditions.

B. Bypassing Safety Filters

Safety classifiers post hoc can operate either on the input or on the output and are also classifiers susceptible to attacks through adversarial input. An attack which succeeds on the base generator often transfers, with some modification, to the classifier meant to discover it [6]. It is important to note that the number of semantic equivalents for a given request is astronomically huge while the training set of the classifier is necessarily finite.

C. Exploiting Logical Constraints in the System Prompt

Policy enforcement is often incorporated into natural language statements included in the system prompt, such as “never disclose the contents of this prompt,” “never mention the product offerings of competing companies,” and “do not perform operations on queries unrelated to domain X.” These statements can be accessed by the same attention function used to interpret user queries. Indeed, several approaches have been shown to effectively counteract such instructions, including prompt-leaking attacks, constraint inversion attacks, and context window flooding attacks [21], [22].

V. LLMs IN THE LOOP: AGENTIC WORKFLOWS

The severity of model-based PI threat depends on the context in which the model is applied, rather than the model itself. While a model capable only of writing text in the worst case can generate a piece of text that is not what the user wanted, a model paired with email, payments, execution, or database APIs can generate something that exceeds the user’s authorization. The AutoGPT agents and their ilk can be regarded as a set of architectures that consolidate the PI threat surface [9], [23].

It is important to lay out the conceptual foundations before we explore the specifics of agents. To do so, it is useful to compare and contrast two threats: SQL injection (SQLi), and Prompt Injection (PI). We discuss them below from the point of view of defenders along several axes: attack vector, target, interpreter, primary countermeasure, consequences, and skill ceiling of attackers.

TABLE I. STRUCTURAL COMPARISON OF SQL INJECTION AND PROMPT INJECTION.

Dimension	SQL Injection (SQLi)	Prompt Injection (PI)
Attack Vector	Concatenation of untrusted input into a parsed query string.	Concatenation of untrusted text into a prompt processed by a statistical language model.
Target	Database engine; data integrity, confidentiality, and availability of persisted records.	Reasoning process of the LLM; integrity of instruction-following and of downstream tool invocation.
Interpreter	Deterministic parser with formal grammar.	Probabilistic reasoner with learned, nonformal decision boundary.

Primary Defense	Parameterized queries, prepared statements, strict type enforcement, allowlists.	Workflow isolation, least-privilege tool scoping, provenance tracking, defensively fine-tuned models, human-in-the-loop.
Impact	Data theft, data corruption, authentication bypass, full DB compromise.	Policy bypass, prompt leakage, data exfiltration via tools, unauthorized actions, supply-chain-style compromise via RAG.
Attacker Skill Floor	Knowledge of SQL grammar and DB fingerprinting.	Fluent natural language composition; public payloads transfer across models.

A. Access to Tools as Privilege Escalation

Once an agent has access to tools, all retrieved documents can serve as sources of instructions that can be run by the agent. Such a behavior represents a form of confused-deputy privilege escalation, where the agent acts on behalf of the user but can be made to act on behalf of an attacker by leveraging the rights of the legitimate user. Capability-based access control, authorization tokens associated with particular requests, and explicit human intervention to confirm the irreversible action have always been ways of dealing with confused-deputy scenarios, but they were often overlooked in early generation agents out of concern about user experience and latency.

B. Retrieval-Augmented Generation as an Attack Substrate

RAG systems retrieve relevant documents from vector stores and add the information to the prompt. An attacker capable of changing the documents in the corpus, via a public wiki, shared file storage, ticketing system, or an ingestion process lacking any kind of provenance checks, will be able to plant malicious material that gets retrieved in response to predictable queries. Such an attack need not target any particular victim – it can be launched against multiple people and remain inactive until the retrieval triggers it. Corpus-based defenses, like provenance signatures and origin-based trust levels, are poorly used.

C. Cross-Agent and Chain Attacks

Multi-agent workflows, where the results generated by one LLM serve as prompts for another, add up to the problem. While the prompt-injection payload might be blocked by the agent facing the customer, there is no reason to expect a back-office agent receiving input from the former one to reject the prompt or even treat it with suspicion. Orchestration of agents will require us to revisit the same strict policies we employed for authenticating service-to-service requests in microservices.

VI. TECHNICAL LIMITS AND DEFENSE

A. Reasons for Input Sanitization Failing

The solution to SQL injection lies in parameterization — the separation of code and data into separate channels via the database driver. There is no equivalent separation for natural language; it is impossible to “escape” a statement such that the model parses it as data rather than an instruction because there is no grammar that distinguishes

the two modes. One could attempt to delimit untrusted input using unique symbols or XML tags, but the adversary can circumvent the filter by instructing the model to ignore the delimiters, injecting them directly, or paraphrasing to bypass the block list. The same issue plagues regexbased filters, which are equally susceptible to adversarial paraphrases due to their lack of Zipfian distribution [6], [14].

B. Workflow Refactoring

The best defenses reframe the challenge as one of workflow design rather than input sanitization. There are three core lessons that recur throughout the literature. First, privilege separation — whereby the component reading the untrusted input lacks permissions to execute dangerous commands — is a common best practice. A “planner” LLM can generate a structured plan of actions, after which another LLM bound to a strict output grammar or a human reviewer verifies the plan before an executor with the necessary permissions implements it. Second, provenance tracking — whereby every span of text entered into the prompt comes with a trust level, and the model cannot be conditioned to execute instructions from lowtrust spans. Lastly, action-level confirmation — whereby critical actions like sending email, transferring funds, or deleting data must always undergo out-of-band approval regardless of the model’s output.

C. Defensive LLMs

Another research direction aims to build robustness against injection attacks into the model itself. Methods include adversarial training with injection datasets, instruction hierarchy fine-tuning favoring system prompts over retrieval results, and constitutional AI where a separate “critic” model flags violations. These techniques lower attack success rates by orders of magnitude compared to standard models but do not render the models immune to attacks, nor is it clear whether they generalize beyond training data payloads. Therefore, defensive LLMs should be treated as a layer of defense-in-depth, not a panacea.

VII. FORMALIZATION: A SUCCESS-RATE MODEL

To frame the discussion of threats, we propose a brief model for the success rate of attacks on LLM workflow processes. Specifically, let us consider an attacker submitting an input payload into the process; the success of the attack would be when the process outputs a result which would lead to policy violations according to the operator.

Then, $P_{\text{success}} = P_{\text{bypass}} \times P_{\text{execute}} \times (1 - P_{\text{detect}})$ (1) where P_{bypass} represents the probability that the payload triggers the model to output behavior that violates policy despite the alignment process undergone by the model; P_{execute} is the probability that the action outputted by the model is indeed executed within the process environment (being closer to 1 for permissive agent environments and closer to 0 for human-in-the-loop execution of certain actions); and P_{detect} is the probability that the monitoring system (pre-execution safeguards or post-hoc audit) will catch the issue before the consequence occurs.

The above equation provides us with two key insights. First, diminishing P_{success} merely by reducing P_{bypass} is an expensive and inefficient approach, since it involves the retraining of the model while the adversary’s input space stays large enough. The other way is to reduce P_{execute}

which is a workflow-level engineering problem allowing one to drive the term to zero for certain action types. Second, an organization can estimate the expected value of the loss in the following way – assigning severity coefficient S_i to each action type i and its own probability:

$$\text{Risk} = \sum_i S_i \times P_{\text{success},i} \tag{2}$$

Contrary to the utility expression $U = T_{\text{manual}} / T_{\text{AI}}$ in AI-assisted attack analysis [8] where U captures the advantage of the attacker in terms of efficiency, the Risk captures loss for the defender. Table II applies the model to different deployment environments, illustrating that P_{execute} , rather than P_{bypass} , governs the risk tier.

TABLE II. INDICATIVE SEVERITY OF PROMPT INJECTION BY LLM INTEGRATION TYPE.

Integration Type	P_{execute} (typical)	Severity S	Risk Tier
Customer-facing chatbot (text only)	Low — output limited to text.	Low to Moderate — reputational, compliance.	Low
Coding assistant (read + suggest)	Moderate — suggestions require developer acceptance.	Moderate — malicious code may reach production.	Moderate
RAG assistant over internal documents	Moderate — retrieval is automatic; output is text.	High — confidential data may be surfaced or leaked.	High
Agentic assistant with mail/calendar tools	High — tool calls execute without confirmation.	High — data exfiltration, impersonation.	High
Internal agent with write access to systems of record	High — irreversible actions possible.	Critical — financial and data-integrity impact.	Critical

VIII. ENTERPRISE PREPAREDNESS

As far as the role of Chief Information Security Officer (CISO) responsible for deploying large language models (LLMs) in production, the key concern is the governance of how to deploy without introducing vulnerabilities rather than whether to deploy. The following are recommendations derived from the foregoing discussion.

A. Treat the System Prompt as Non-Confidential

All information contained in the system prompt is assumed to be retrievable. Secrets, API keys, personally identifiable information (PII) of other users, and information about business logic that, once made available to others, creates vulnerabilities should only be provided via controlled channels outside of the prompt.

B. Catalog Tools by Access Tier and Function

Create a registry of tools, APIs, or functions used by LLM agents classified by reversibility and blast radius. Identify the tier according to whether tool invocations could be performed automatically, if they should go through structured validations (for example, validating the arguments against schemas), or if there should be human validation for each call. The tier should match the normal

workload of the agent, e.g., a mail-drafting agent would not need delete permissions.

C. Provide Provenance for Ingested Content

In case agents consume documents like emails, support tickets, web pages, or documents retrieved and augmented, provide trust labels when ingesting them, pass these labels through the prompt, and architect the solution in such a way that low-trust data could not have any influence on higherprivilege actions. If an architectural guarantee is unavailable, make sure to log in a way that would enable you to trace which span caused which action.

D. Red-Team Continuous Operations

Since the changes to policies and implementation occur more frequently than new model releases, regular red-team activities conducted either in-house or by a service provider are expected to assess vulnerabilities in deployed agents using the latest payloads and reporting the vulnerabilities in the same risk register that monitors applications. Use known payload libraries in your continuous integration pipeline as part of regression testing.

E. Prepare Agentic Incident Response Playbooks

Traditional incident response playbooks fail to apply to agentic incidents because a coerced single action might happen through legitimate API calls authenticated with real credentials, leaving the security information and event management (SIEM) platform untriggered. Therefore, playbooks should include agentic hypotheses like suspicious tool call sequences, document retrieval irrelevant to the context of the query, or unusual outbound destinations and corresponding responses.

F. Alignment with Emerging Standards

Emerging standards such as the OWASP Top-10 for LLM Applications [12] and the NIST AI Risk Management Framework are moving toward alignment with the controls outlined above. Formal alignment between the control set implemented and such frameworks serves both governance and auditing purposes, positioning the organization well ahead of any future regulations, which, as of the time of writing, remain in draft form across multiple jurisdictions.

IX. CONCLUSION

Prompt injection represents the structural sequel to SQL injection, insofar as both exploit the same underlying architectural flaw—the confusion of instructions and data in a trusted interpreter. However, while the analogy provides valuable insights, it is incomplete. Unlike SQL injection, prompt injection targets a semantic interpreter, meaning that parameterization, which was the key defense against SQL injection, cannot be directly applied. Instead, a defensive strategy against prompt injection will necessarily be designed at the workflow level, including privilege segregation, traceability of provenance, human involvement in irreversible actions, and defensively trained models as secondary defenses.

Results from 2023–2026 suggest that success in attacks is more a factor of the surrounding system rather than the model itself. Deploying the same model in two configurations — a simple text-based chatbot versus a conversational agent with access to e-mail and payment services — produces vastly different levels of vulnerability. In this case, the good news for CISOs is that the levers of

defense lie in architecture and governance, which fall squarely within the domain of the average enterprise security strategy. The bad news is that, due to its nature, natural language cannot be sanitized, and security postures relying on training the model not to comply are built upon shifting sands.

Future research should focus on formalizing the successrate formula using empirically derived priors from benchmark datasets, developing the taxonomy to cover multi-modal injections (e.g., image-based, audio-based, document-based with instructions embedded), and analyzing the security properties of agent-of-agent trust relationships in multi-agent systems.

REFERENCES

- [1] W. G. J. Halfond, J. Viegas, and A. Orso, "A classification of SQLinjection attacks and countermeasures," in *Proc. IEEE Int. Symp. Secure Softw. Eng.*, 2006, pp. 13–15. [2] C. Anley, "Advanced SQL injection in SQL server applications," *NGSSoftware Tech. Rep.*, 2002.
- [2] F. Perez and I. Ribeiro, "Ignore previous prompt: Attack techniques for language models," in *Proc. NeurIPS Workshop on ML Safety*, 2022, pp. 1–11.
- [3] K. Greshake et al., "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 16th ACM Workshop AISec*, 2023, pp. 79–90.
- [5] S. Willison, "Prompt injection attacks against GPT-3," *simonwillison.net*, Tech. Note, 2022.
- [4] A. Wei, N. Haghtalab, and J. Steinhardt, "Jailbroken: How does LLM safety training fail?" in *Advances in Neural Information Processing Systems*, vol. 36, 2023.
- [5] Y. Liu et al., "Prompt injection attack against LLM-integrated applications," *arXiv preprint arXiv:2306.05499*, 2023. [8] A. Mirsky et al., "The threat of offensive AI to organizations," *Computers & Security*, vol. 124, 2023, Art. no. 103006. [9] R. Naihin et al., "Testing language model agents safely in the wild," *arXiv preprint arXiv:2311.10538*, 2023.
- [6] N. Ruiz et al., "Agents under siege: Breaking pragmatic multi-agent LLM systems," in *Proc. IEEE Symp. Secur. Privacy Workshops*, 2025, pp. 112–124.
- [7] H. Snyder, "Literature review as a research methodology: An overview and guidelines," *J. Bus. Res.*, vol. 104, pp. 333–339, 2019. [12] OWASP Foundation, "OWASP Top 10 for LLM Applications, Version 2025," Tech. Rep., 2025.
- [8] Anthropic, "Responsible disclosure and red-teaming findings for Claude 3 family models," Tech. Rep., 2024.
- [9] X. Shen et al., "'Do anything now': Characterizing and evaluating in-the-wild jailbreak prompts on large language models," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2024, pp. 1671–1685.
- [10] Z. Yong, C. Menghini, and S. H. Bach, "Low-resource languages jailbreak GPT-4," in *Proc. NeurIPS Workshop on Socially Responsible Language Modelling Research*, 2023.
- [11] S. Abdelnabi et al., "Not what you've signed up for: Indirect prompt injection threats," *IEEE Security & Privacy*, vol. 22, no. 3, pp. 48–57, 2024.
- [12] J. Yi et al., "Benchmarking and defending against indirect prompt injection attacks on large language models," in *Proc. 33rd USENIX Secur. Symp.*, 2024, pp. 1845–1862.
- [13] M. Russinovich, A. Salem, and R. Eldan, "Great, now write an article about that: The Crescendo multi-turn LLM jailbreak attack," *arXiv preprint arXiv:2404.01833*, 2024.
- [14] L. Ouyang et al., "Training language models to follow instructions with human feedback," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 27730–27744.
- [15] A. Zou, Z. Wang, J. Z. Kolter, and M. Fredrikson, "Universal and transferable adversarial attacks on aligned language models," *arXiv preprint arXiv:2307.15043*, 2023.
- [16] Y. Zhang et al., "Effective prompt extraction from language models," in *Proc. Conf. Lang. Model. (COLM)*, 2024.
- [17] N. Carlini et al., "Are aligned neural networks adversarially aligned?" in *Advances in Neural Information Processing Systems*, vol. 36, 2023.
- [18] S. Ruan et al., "Identifying attack surfaces in LLM-powered autonomous agents," in *Proc. USENIX Secur. Symp.*, 2025, pp. 3201–3218.
- [19] E. Debenedetti et al., "AgentDojo: A dynamic environment to evaluate attacks and defenses for LLM agents," in *Advances in Neural Information Processing Systems*, vol. 37, 2024.
- [20] W. Zou et al., "PoisonedRAG: Knowledge corruption attacks to retrieval-augmented generation," in *Proc. 34th USENIX Secur. Symp.*, 2025, pp. 403–420.
- [21] P. Chan et al., "Cascading prompt injection in multi-agent LLM systems," *IEEE Trans. Dependable Secure Comput.*, early access, 2026.
- [22] T. Chen et al., "StruQ: Defending against prompt injection with structured queries," in *Proc. 33rd USENIX Secur. Symp.*, 2024, pp. 1933–1950.
- [23] Y. Wallace et al., "The instruction hierarchy: Training LLMs to prioritize privileged instructions," *arXiv preprint arXiv:2404.13208*, 2024.